

MSAL Authentication Explained

React + Microsoft Entra ID + OAuth 2.0 / OpenID Connect

Core idea: OAuth 2.0 is the authorization protocol, Microsoft Entra ID is the identity platform, and MSAL is Microsoft's library that helps your React application perform authentication and acquire tokens.

1. The Basic Architecture

Imagine you are building an Employee Portal. Only employees from your company should be able to sign in. The basic flow is:

```
User -> React Application -> MSAL -> Microsoft Entra ID
|
v
User authenticates
|
v
React Application <- Tokens <- MSAL
|
| Access Token
v
Backend API
```

2. What is Microsoft Entra ID?

Microsoft Entra ID (previously Azure Active Directory / Azure AD) is Microsoft's cloud identity platform. An organization can have its users, groups, applications, permissions, MFA policies, and other identity information inside its own directory.

```
Microsoft Entra ID
|
+-- ABC Company Tenant
|   +-- Users
|   +-- Groups
|   +-- Applications
|
+-- XYZ Company Tenant
    +-- Users
    +-- Applications
```

3. What is a Tenant?

A tenant is an organization's dedicated identity directory in Microsoft Entra ID. For example, ABC Company can have one tenant containing its employees, groups, security policies, and registered applications.

4. What is the Tenant ID?

Every tenant has a unique identifier called the Tenant ID. Think of it as answering the question: Which organization's directory should Microsoft use for authentication?

```
ABC Company
Tenant ID: 7ab12345-12ab-45cd-89ef-123456789abc
```

5. Why Create an App Registration?

Microsoft may know your organization, but it also needs to know which application is requesting authentication. An App Registration registers your application with Microsoft Entra ID. For example, you could register an application named Employee Portal and configure it as a Single-Page Application (SPA).

```
ABC Company Tenant
|
+-- App registrations
+-- Employee Portal
+-- HR Portal
+-- Finance Portal
```

6. What is the Client ID?

When you create an App Registration, Microsoft generates an Application (client) ID. The Client ID uniquely identifies that particular application. Multiple applications can belong to the same tenant, but each application has its own Client ID.

```
ABC Company - Tenant ID: 111111
|
+-- Employee Portal - Client ID: AAA
+-- HR Portal - Client ID: BBB
+-- Finance Portal - Client ID: CCC
```

7. Tenant ID vs Client ID

The easiest way to remember the difference is:

Tenant ID = Which organization?

Client ID = Which application?

8. What is the Redirect URI?

Your React application sends the browser to Microsoft's login system. After authentication, Microsoft needs to know where it is allowed to send the browser back. That location is the Redirect URI. For local development it might be `http://localhost:5173`; production might use your real application domain. The URI must be registered in Entra ID for security.

9. MSAL Configuration

A typical MSAL configuration for a React SPA looks like this:

```
const msalConfig = {
  auth: {
    clientId: "YOUR_CLIENT_ID",
    authority:
      "https://login.microsoftonline.com/YOUR_TENANT_ID",
    redirectUri: window.location.origin
  }
};
```

10. What Does This Configuration Mean?

Read the configuration in plain English as: My application (Client ID) wants Microsoft to authenticate users using this organization's directory (Tenant ID), and after authentication

Microsoft should return the browser to my application (Redirect URI).

11. PublicClientApplication

In a browser application you normally create a `PublicClientApplication` instance. A React SPA is a public client because code running in the browser cannot safely keep a secret.

```
const pcaInstance =  
  new PublicClientApplication(msalConfig);
```

12. User Clicks Login

When the user clicks Login, your application can call `loginRedirect()`. MSAL builds the authorization request and redirects the browser to Microsoft's login system.

```
await pcaInstance.loginRedirect({  
  scopes: ["openid", "profile", "User.Read"]  
});
```

13. What Microsoft Receives

Conceptually, the request tells Microsoft:

- Which application is requesting login - Client ID
- Which tenant should authenticate the user - Tenant ID / authority
- Which permissions are requested - scopes
- Where the browser may return - Redirect URI

14. Microsoft Authenticates the User

The user enters credentials on Microsoft's login page and may complete MFA. Your React application does not receive the user's Microsoft password. Microsoft performs the authentication.

15. Why Code After `loginRedirect()` Does Not Behave Like a Normal Function Call

With redirect authentication, the browser navigates away from your React application. The current page is unloaded. After authentication, Microsoft redirects the browser back and your application loads again. Therefore, post-login processing belongs in the redirect-handling/application-initialization flow rather than assuming normal synchronous continuation after `loginRedirect()`.

16. Authorization Code Flow + PKCE

Modern MSAL browser applications use the OAuth 2.0 Authorization Code Flow with PKCE. At a high level, MSAL sends an authorization request, Microsoft authenticates the user, an authorization code is returned, and MSAL securely completes the exchange for tokens. MSAL handles most of these details for you.

```
React/MSAL -> Microsoft -> User Login  
^ |  
| v  
+--- Authorization Code ---+  
|
```

```
+--- Token exchange with PKCE
|
+--- Tokens
```

17. What Does MSAL Store in Session Storage?

Depending on your MSAL cache configuration, you may see MSAL-related entries in browser storage after authentication. These can represent account information, tokens, request state, scopes, expiration information, and other cache metadata. Normally, you should not manually read or modify these entries. Use MSAL APIs instead.

```
const accounts = pcaInstance.getAllAccounts();
console.log(accounts);
```

18. Authentication is Complete - What Next?

Signing the user in answers the identity question, but your React application may still need permission to call a protected backend API. For that, it requests an access token intended for that API.

19. `acquireTokenSilent()`

Once a user is authenticated, use MSAL to obtain an appropriate access token without unnecessarily interrupting the user. `acquireTokenSilent()` checks/uses MSAL's token cache and performs silent token acquisition when possible.

```
const response = await pcaInstance.acquireTokenSilent({
  scopes: apiRequest.scopes,
  account: pcaInstance.getAllAccounts()[0]
});

const accessToken = response.accessToken;
```

20. Calling Your Backend API

The React application sends the access token to the backend using the HTTP Authorization header.

```
fetch("https://api.company.com/employees", {
  headers: {
    Authorization: `Bearer ${accessToken}`
  }
});
```

21. What Does the Backend Do?

The backend should validate the access token and enforce authorization. A successful login in the browser is not enough by itself. The API must verify that the token is valid and intended for the API and that the caller has the required permissions.

```
React
|
| Authorization: Bearer <access-token>
v
Backend API
|
+-- Validate token
```

```
+-- Check audience / issuer
+-- Check scopes or roles
|
+-- Valid -> Return protected data
+-- Not valid -> Reject request
```

22. ID Token vs Access Token

An ID token is primarily about the authenticated user's identity - think Who is this user? An access token is intended for a protected resource/API - think What resource is this token authorized to access? Do not use an ID token as a substitute for a properly issued API access token.

23. What are Scopes?

Scopes represent permissions that an application requests for a resource. For example, an Employee API might expose a permission such as `Employees.Read`. Your React application requests that scope when acquiring an access token, and the API can enforce the corresponding permission.

```
const apiRequest = {
  scopes: [
    "api://BACKEND_CLIENT_ID/Employees.Read"
  ]
};
```

24. Why No Client Secret in React?

Never place a client secret in React code or a Vite client environment variable. Browser code is delivered to the user's machine and can be inspected. A SPA is therefore a public client. PKCE allows the authorization-code flow to be used securely without pretending that the browser can keep a client secret.

```
// Do NOT do this in React:
const clientSecret = "ABC123SECRET";

// Also NOT safe:
VITE_CLIENT_SECRET=ABC123SECRET
```

Complete Flow - One Picture

The complete mental model is: your organization owns an Entra tenant; your React application is registered inside that tenant; the App Registration gives the application a Client ID; MSAL uses the Client ID, tenant/authority, redirect URI, and scopes to authenticate the user and acquire tokens; your React application then uses an access token to call a protected API.

```
1. User opens React app
|
2. User clicks Login
|
3. MSAL loginRedirect()
|
4. Browser goes to Microsoft Entra ID
|
5. Tenant ID identifies the organization's directory
|
6. Client ID identifies the React application
|
7. User authenticates with Microsoft (and MFA if required)
|
8. Microsoft redirects browser to registered Redirect URI
|
9. MSAL completes Authorization Code + PKCE flow
|
10. MSAL caches account/token information
|
11. React asks MSAL for an access token
|
12. React calls backend with Bearer access token
|
13. Backend validates token + permissions
|
14. Backend returns protected data
```

Quick Reference

Term	Think of it as
Microsoft Entra ID	Microsoft's identity and authentication platform
Tenant	Your organization's identity directory
Tenant ID	Which organization/directory?
App Registration	Registers your application with Entra ID
Client ID	Which application?
Redirect URI	Where Microsoft may send the browser after authentication
MSAL	Library that manages authentication and token acquisition
ID Token	Identity information - who is the user?
Access Token	Token intended for a protected API/resource

Term	Think of it as
Scope	A permission requested for a resource/API
PKCE	Protection used with authorization-code flow for public clients

Memory trick: Tenant ID = company. Client ID = application. Redirect URI = where to return. MSAL = the library doing the authentication/token work. Access token = what the frontend sends to the protected API.